CS3307 Final Submission

Timely

A Calendar Application

Bryan Jang & Daniel Yuan

Contact Info for Bryan Jang
251251859
bjang26@uwo.ca

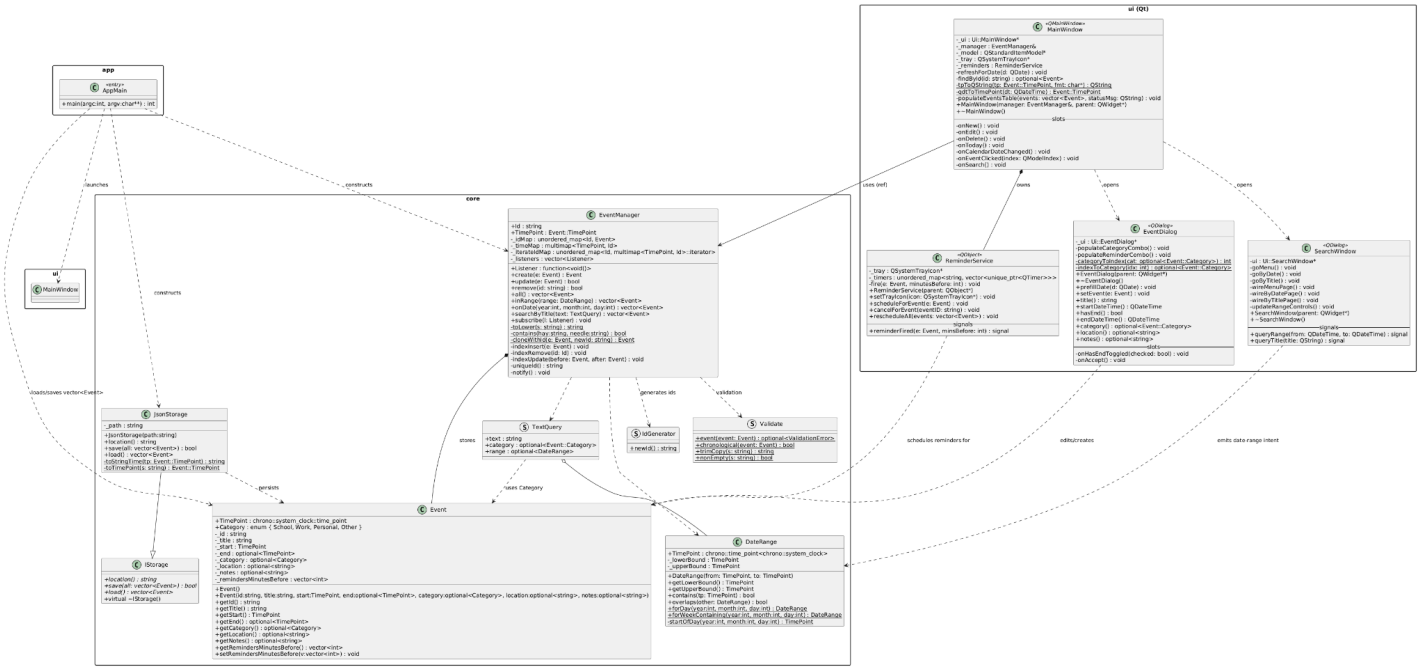
Contact info for Daniel Yuan
251301805
dyuan42@uwo.ca

Design Documentation

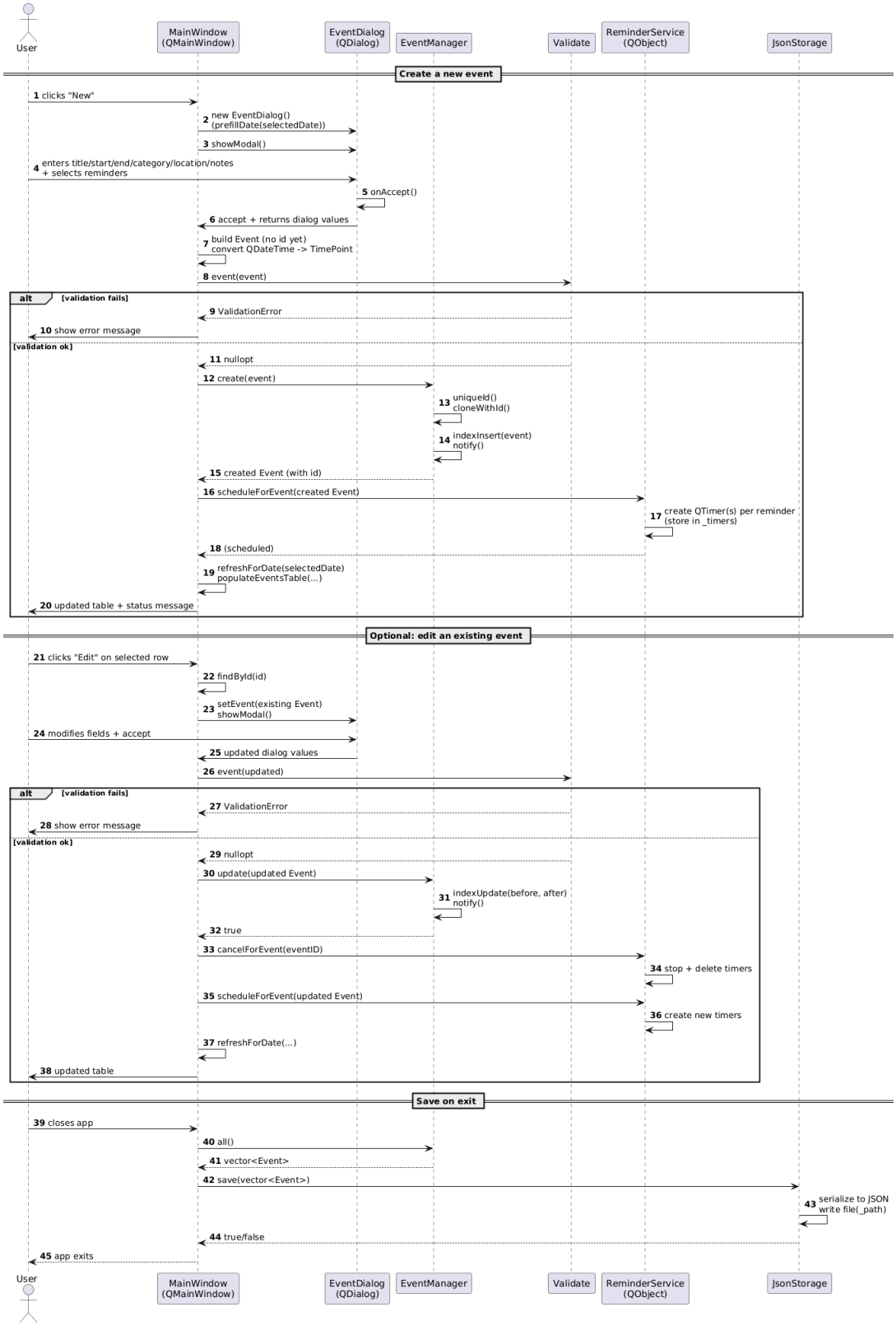
System overview

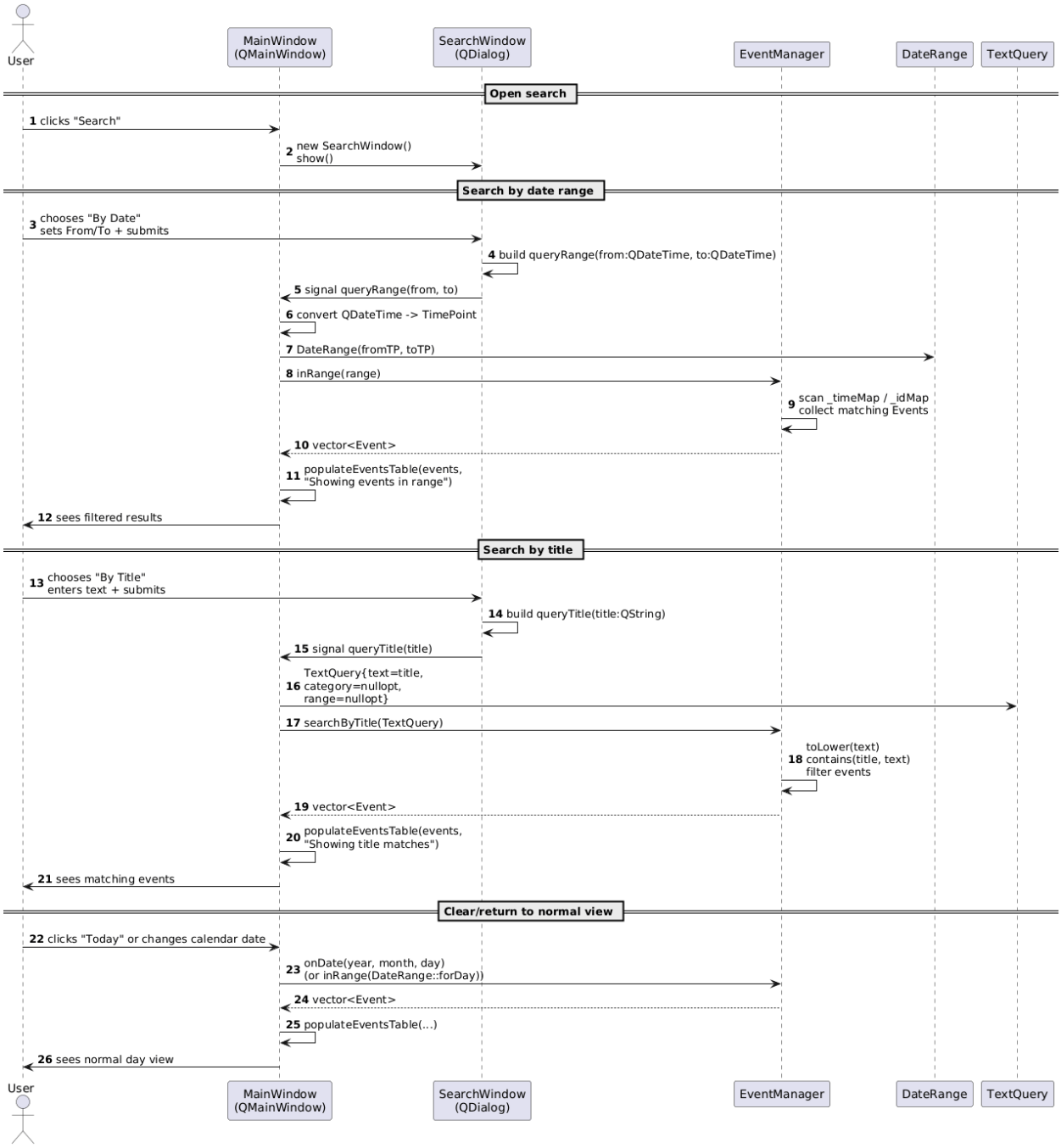
This project is a Qt (C++) desktop calendar application that lets a user browse dates, view the events scheduled for a selected day, and create/edit/delete events through dialog-driven forms. The core architecture separates UI presentation (Qt widgets and dialogs) from domain logic (event objects and validation) and persistence (loading/saving the event set), so the calendar view stays responsive while the event data remains consistent.

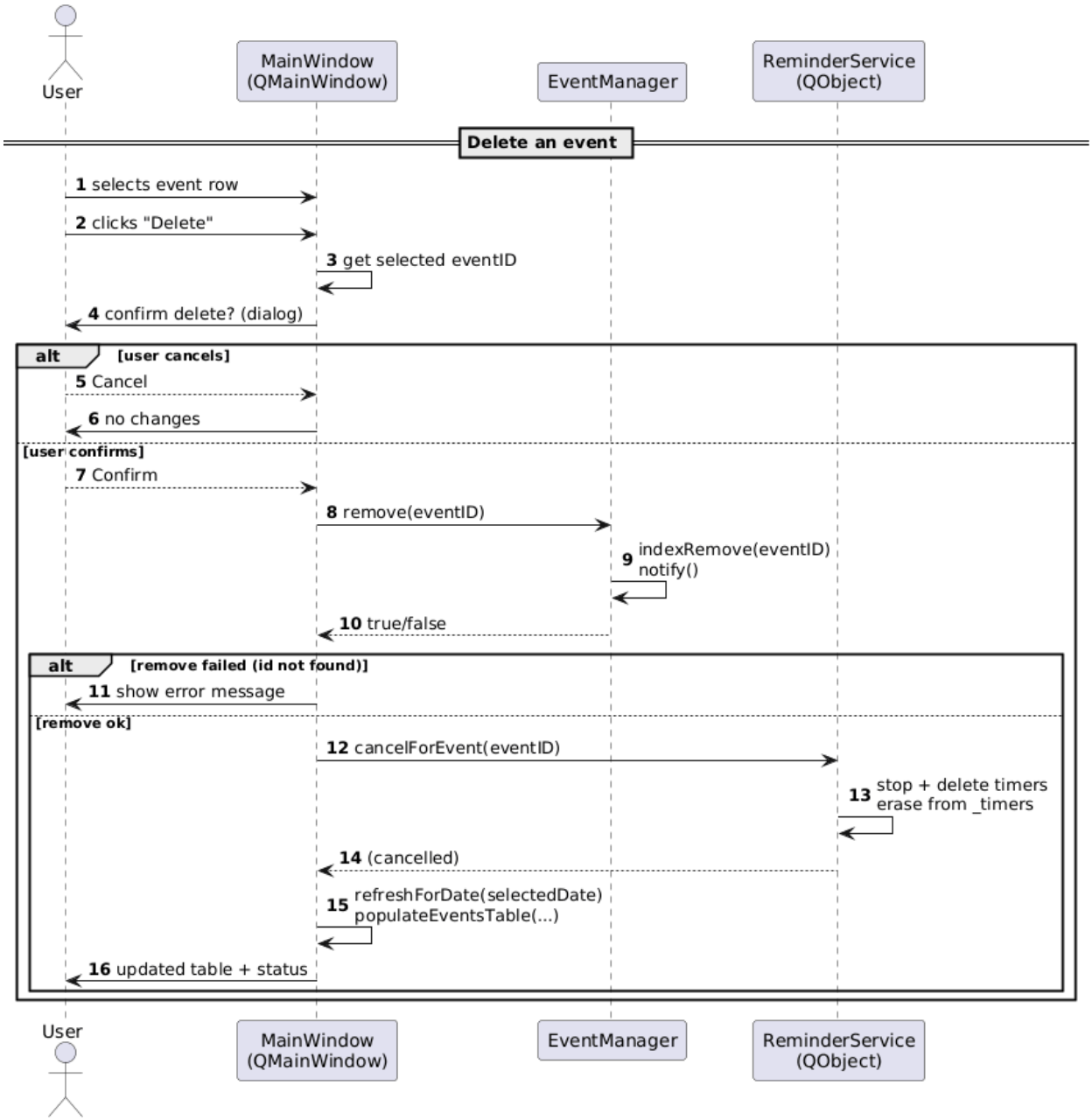
Class Diagrams

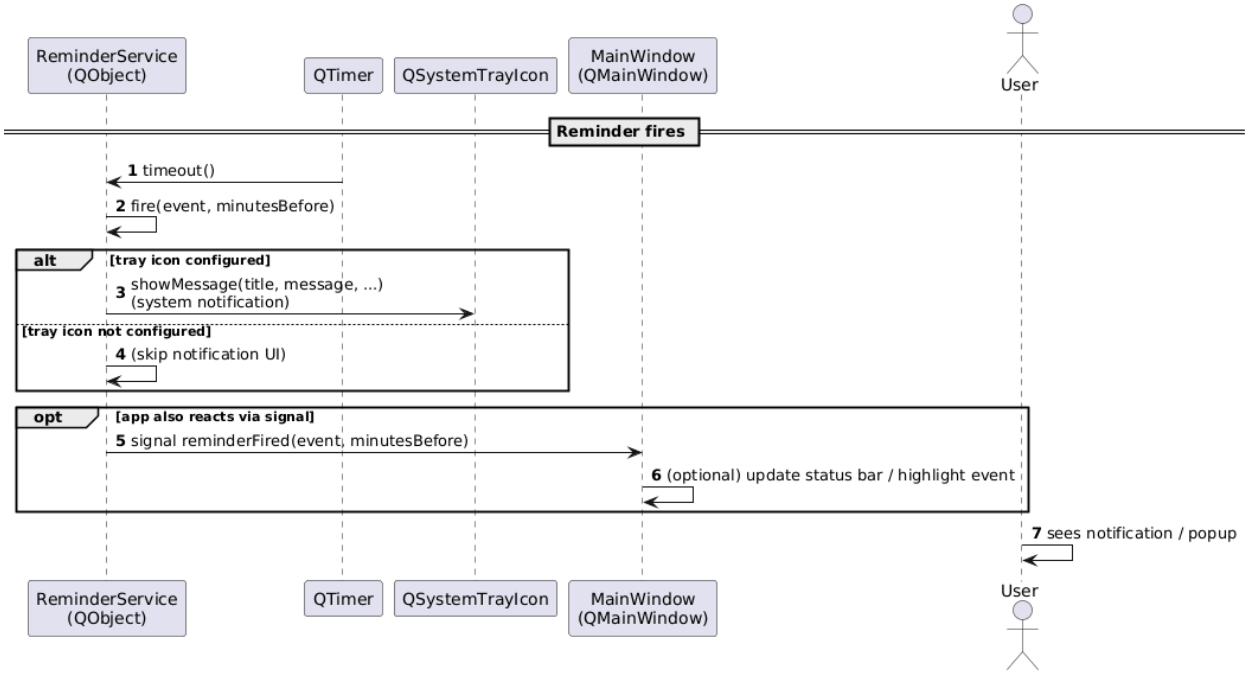


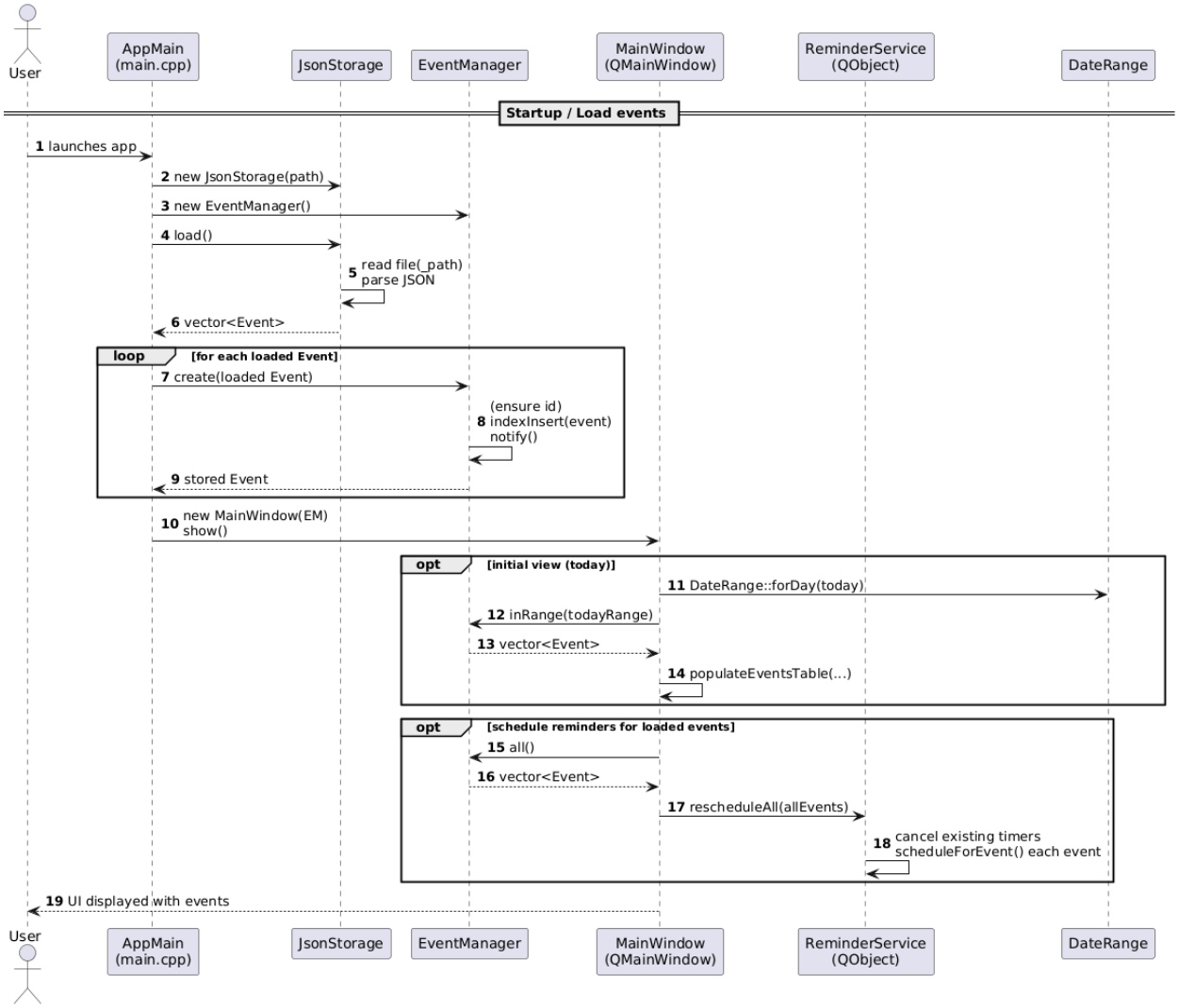
Sequence Diagrams



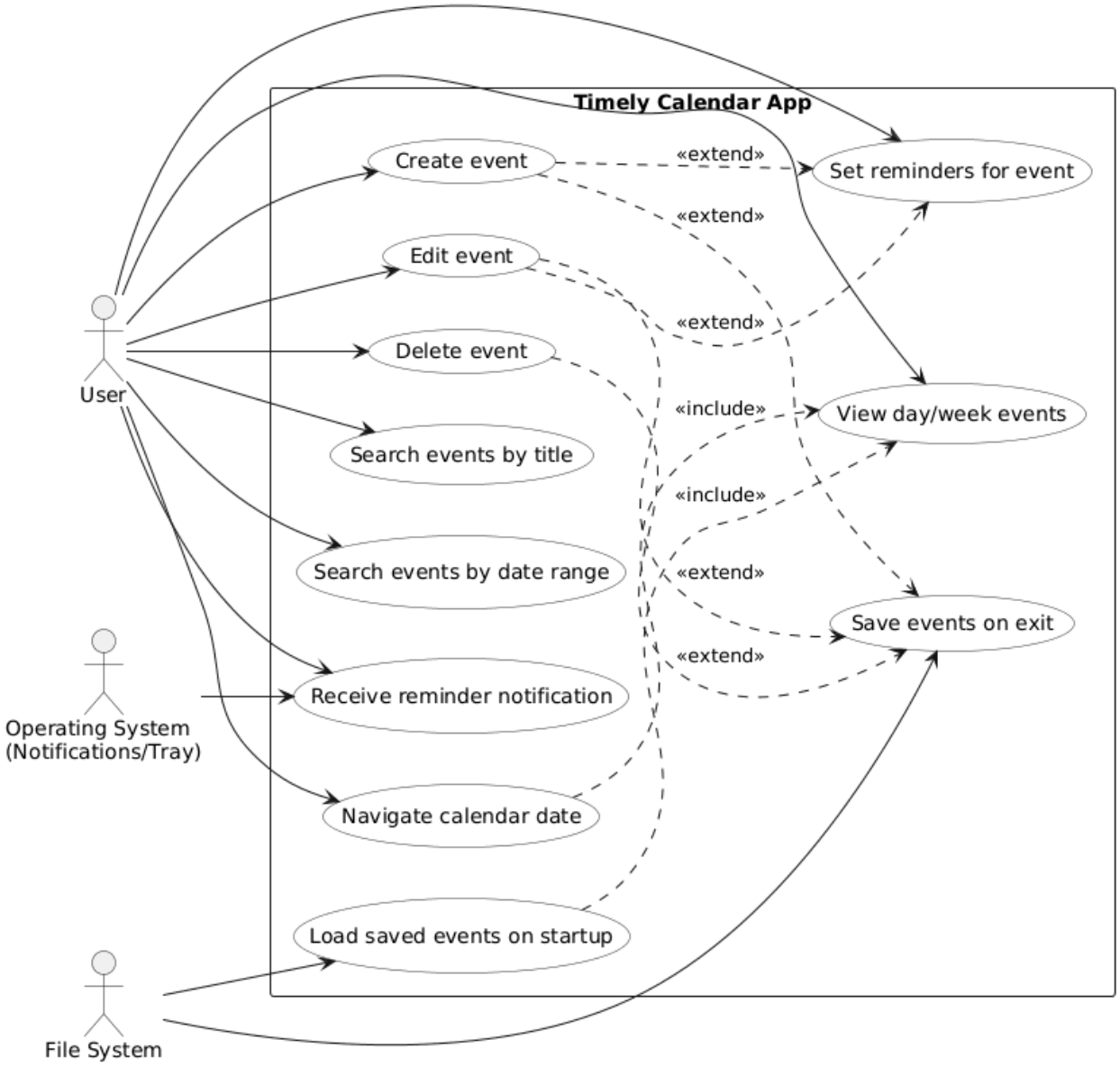

User
MainWindow (QMainWindow)
SearchWindow (QDialog)
EventManager
DateRange
TextQuery
Open search
1 clicks "Search"
2 new SearchWindow() show()
Search by date range
3 chooses "By Date" sets From/To + submits
4 build queryRange(from:QDateTime, to:QDateTime)
5 signal queryRange(from, to)
6 convert QDateTime -> TimePoint
7 DateRange(fromTP, toTP)
8 inRange(range)
9 scan _timeMap / _idMap collect matching Events
10 vector<Event>
11 populateEventsTable(events, "Showing events in range")
12 sees filtered results
Search by title
13 chooses "By Title" enters text + submits
14 build queryTitle(title:QString)
15 signal queryTitle(title)
16 TextQuery{text=title, category=nullopt, range=nullopt}
17 searchByTitle(TextQuery)
18 toLower(text) contains(title, text) filter events
19 vector<Event>
20 populateEventsTable(events, "Showing title matches")
21 sees matching events
Clear/return to normal view
22 clicks "Today" or changes calendar date
23 onDate(year, month, day) (or inRange(DateRange::forDay))
24 vector<Event>
25 populateEventsTable(...)
26 sees normal day view
User
MainWindow (QMainWindow)
SearchWindow (QDialog)
EventManager
DateRange
TextQuery


User
MainWindow (QMainWindow)
EventManager
ReminderService (QObject)
Delete an event
1 selects event row
2 clicks "Delete"
3 get selected eventID
4 confirm delete? (dialog)
alt
[user cancels]
5 Cancel
6 no changes
[user confirms]
7 Confirm
8 remove(eventID)
9 indexRemove(eventID)
notify()
10 true/false
alt
[remove failed (id not found)]
11 show error message
[remove ok]
12 cancelForEvent(eventID)
13 stop + delete timers
erase from _timers
14 (cancelled)
15 refreshForDate(selectedDate)
populateEventsTable(...)
16 updated table + status
User
MainWindow (QMainWindow)
EventManager
ReminderService (QObject)


ReminderService
(QObject)
QTimer
QSystemTrayIcon
MainWindow
(QMainWindow)
User
Reminder fires
1 timeout()
2 fire(event, minutesBefore)
alt
[tray icon configured]
3 showMessage(title, message, ...)
(system notification)
[tray icon not configured]
4 (skip notification UI)
opt
[app also reacts via signal]
5 signal reminderFired(event, minutesBefore)
6 (optional) update status bar / highlight event
7 sees notification / popup
ReminderService
(QObject)
QTimer
QSystemTrayIcon
MainWindow
(QMainWindow)
User


User
AppMain (main.cpp)
JsonStorage
EventManager
MainWindow (QMainWindow)
ReminderService (QObject)
DateRange
Startup / Load events
1 launches app
2 new JsonStorage(path)
3 new EventManager()
4 load()
5 read file(_path) parse JSON
6 vector<Event>
loop [for each loaded Event]
7 create(loaded Event)
8 (ensure id) indexInsert(event) notify()
9 stored Event
10 new MainWindow(EM) show()
opt [initial view (today)]
11 DateRange::forDay(today)
12 inRange(todayRange)
13 vector<Event>
14 populateEventsTable(...)
opt [schedule reminders for loaded events]
15 all()
16 vector<Event>
17 rescheduleAll(allEvents)
18 cancel existing timers scheduleForEvent() each event
19 UI displayed with events

Use Case Diagrams



System Modules

1. User Interface Layer (Presentation)

MainWindow

The **MainWindow** class acts as the central UI controller for the application

Responsibilities:

- Displays the main calendar (**QCalendarWidget**) for date selection
- Displays daily events in a **QTableView (eventsView)**
- Handles toolbar actions (New, Edit, Delete, Today)
- Responds to user interactions (date changes, event clicks)
- Coordinates between UI actions and the **EventManager**

Key UI Elements:

- **QCalendarWidget calendar** — selects the active date
- **QTableView eventsView** — shows events for the selected date
- **QStandardItemModel** — backs the event table
- **QToolBar** actions — trigger CRUD operations

EventDialog

The **EventDialog** class provides a modal dialog for creating and editing events

Responsibilities:

- Collects event details from the user (title, date/time, category, location, notes)
- Performs input validation (required title, valid time range)
- Converts UI input into structured data for **MainWindow**

Key UI Elements:

- **QLine** for title and location
- **QDateTime** for start and end times
- **QCheckBox** to enable/disable end time
- **QComboBox** for event category
- **QPlainTextEdit** for notes

2. Application / Control Layer

MainWindow Control Logic

MainWindow also functions as the controller that translates UI events into application actions:

- Converts between Qt time types and **std::chrono** time points
- Maps selected table rows back to domain events using stored event IDs
- Subscribes to **EventManager** updates to automatically refresh the UI
- Handles conditional logic such as only showing notes when they exist

This logic ensures that UI components remain thin and do not directly manage event storage or business rules

.3. Domain Layer (Core Data Model)

Event

The **Event** class represents a single calendar event (used throughout the system).

Key attributes include:

- Unique identifier (**id**)
- Title

- Start time (**TimePoint**)
- Optional end time
- Optional category
- Optional location
- Optional notes

Events are immutable once constructed; editing is performed by creating a new **Event** instance with the same ID and updated fields.

4. Event Management Layer

EventManager

The **EventManager** acts as the central authority for event data.

Responsibilities:

- Stores all events in memory
- Assigns unique IDs to new events
- Provides queries such as **onDate(year, month, day)**
- Supports create, update, and delete operations
- Notifies subscribers (e.g., **MainWindow**) when data changes

This design decouples the UI from the underlying data storage mechanism.

Data Flow

1. Application Startup

1. **MainWindow** is constructed with a reference to EventManager
2. UI widgets are initialized and signals are connected

3. **EventManager** subscription is registered
4. Today's date is selected and events are loaded into the table view

2. Viewing Events by Date

1. User selects a date in the calendar
2. **onCalendarDateChanged()** is triggered
3. **MainWindow** calls **EventManager::onDate(...)**
4. Returned events populate the **QStandardItemModel**
5. The **QTableView** updates automatically

3. Creating an Event

1. User clicks **New**
2. **EventDialog** opens with prefilled date
3. User enters details and clicks OK
4. Dialog validates input
5. **MainWindow** constructs an **Event**
6. **EventManager::create()** stores the event
7. Subscribers refresh the event list

4. Editing an Event

1. User selects an event row and clicks **Edit**
2. Event ID is retrieved from the table model (**Qt::UserRole**)
3. Existing event is loaded into **EventDialog**

4. User modifies fields and confirms
5. Updated event replaces the original in **EventManager**
6. UI refreshes automatically

5. Deleting an Event

1. User selects an event and clicks **Delete**
2. Confirmation dialog is shown
3. **EventManager::remove(id)** deletes the event
4. Event list refreshes

6. Viewing Event Notes

1. User clicks an event in the table
2. **onEventClicked()** retrieves the event by ID
3. If notes exist, they are displayed in a message box
4. If no notes exist, no UI action occurs

Design Patterns

Observer (Publish–Subscribe) Pattern

The Observer pattern is used between the **EventManager** and the **MainWindow**.
MainWindow subscribes to the **EventManager** so that whenever events are created, updated, or deleted, the user interface automatically refreshes to reflect the latest data.

Why it was chosen:
This pattern was selected to decouple the event storage logic from the user interface and to ensure the UI always stays synchronized with the underlying event data.

How it improved the system:
It eliminated the need for manual refresh calls after every operation, reduced duplicated code, and ensured consistent UI updates whenever event data changes.

Model–View Pattern (Qt Model/View Architecture)

The application uses Qt's Model–View architecture, where **QStandardItemModel** acts as the model and **QTableView (eventsView)** acts as the view.

Why it was chosen:
Qt's model/view system provides a clean and scalable way to display tabular data, making it well suited for displaying daily events in a calendar application.

How it improved the system:
It cleanly separated data representation from visual presentation, simplified event selection handling, and made the UI easier to extend or modify without changing the underlying data logic.

MVC-Style Separation of Concerns

The system follows an MVC-style structure:

- **Model: Event, EventManager**
- **View: MainWindow.ui, EventDialog.ui**
- **Controller: MainWindow** slot functions

Why it was chosen:
Separating responsibilities prevents UI code from becoming tightly coupled to business logic and data storage.

How it improved the system:
It improved maintainability, made the code easier to understand, and allowed individual components (such as dialogs or storage mechanisms) to be changed independently.

Encapsulation of Object Creation (Builder-like Construction)

Event creation and updates are centralized in **MainWindow**, which constructs **Event** objects using validated input from **EventDialog**.

Why it was chosen:
Centralizing object construction ensures consistent handling of optional fields and time conversions.

How it improved the system:
It reduced duplication, prevented inconsistent event creation, and ensured event identity is preserved during edits.

Dependency Injection

Passes core services into UI components via constructors and setters, and decouples components through Qt signals/slots, preventing classes from directly instantiating or depending on concrete implementations."

Constructor injection:

MainWindow receives an **EventManager** through its constructor instead of creating one itself, keeping UI logic decoupled from event management.

Setter injection:

ReminderService receives a **QSystemTrayIcon** via **setTrayIcon()** rather than constructing it internally, allowing notifications to be swapped or omitted.

Event-driven injection (Qt signals/slots):

SearchWindow emits query signals (**queryDate**, **queryRange**, etc.), and the handling logic is injected by connecting these signals externally, avoiding direct dependencies on **EventManager**.

Result:

Object creation is centralized, coupling is reduced, and components remain flexible, testable, and easier to extend.

Prototype (clone/copy-based creation)

EventManager::cloneWithId(const Event& e, const string& newId)

Builds a new **Event out{ newId, ... }** by copying fields from **e**, then returns it.

That's the Prototype idea: "make a new object based on an existing one," with slight modifications.

Testing Report

1. Testing Approach

Unit testing was used to make sure the core logic of the Timely calendar application works correctly. The tests were written using **GoogleTest**, and **Google Mock** was used where interaction behavior needed to be verified (such as checking whether listeners were notified).

The focus of testing was on the core layer, since it contains the main logic of the application and does not depend on the Qt user interface. Testing the core separately makes the code easier to verify and reduces the chance of bugs affecting the UI.

Two main classes were tested:

- Validate
- EventManager

2. Testing the Validate Class

The Validate class handles checking whether event data is valid before it is added to the system.

The following behaviors were tested:

- Whitespace is trimmed from user input
- Empty or whitespace-only titles are rejected
- Events without an end time are allowed
- Events where the end time is before the start time are rejected

These tests help ensure that invalid events cannot be created and that common input mistakes are handled safely.

3. Testing the EventManager Class

The EventManager class is responsible for creating, updating, deleting, searching, and ordering events. It also notifies listeners when changes occur.

To test this behavior, **Google Mock** was used to create a mock listener. This allowed the tests to check whether notifications were triggered at the correct times using EXPECT_CALL.

Mock listener example

```
struct MockListener {

    MOCK_METHOD(void, onChanged, (), ());

};
```

The mock listener was connected to the EventManager using a lambda that calls the mock method.

4. Using EXPECT_CALL

EXPECT_CALL was used to verify side effects, not just return values.

Examples include:

- Making sure listeners are notified when an event is successfully created
- Making sure listeners are **not** notified when event creation fails
- Confirming listeners are notified again when an event is updated

This ensures that the observer behavior of EventManager is correct and consistent.

5. Tested Scenarios

The following cases were covered by the tests:

Event creation

- A valid event is created successfully
- A missing event ID is automatically generated
- Listeners are notified after creation

Invalid event creation

- Creating an invalid event throws an exception
- No events are added to the manager

- Listeners are not notified

Event updating

- Updating an event's start time correctly reorders events
- Listeners are notified after the update

Searching

- Searches are case-insensitive
- An empty search string returns all events

6. Test Results

```
Test summary:  14 passes, 0 fails.  (80 ms)
PASS   Executing test suite ValidateTest                                                    0 ms
  PASS   ValidateTest.NonEmpty_TrimsWhitespace                                              tst_validate.cpp   7
  PASS   ValidateTest.Chronological_AllowsNoEndAndEndAfterStart                             tst_validate.cpp   13
  PASS   ValidateTest.Event_FailsWhenEndBeforeStart_AndWhenTitleEmpty                       tst_validate.cpp   30
         Test execution took 0 ms total.                                                    tst_validate.cpp   30
PASS   Executing test suite EventManagerTest                                                0 ms
  PASS   EventManagerTest.Create_ValidEvent_AssignsIdIfMissingAndNotifiesListeners          tst_event_manager.cpp 20
  PASS   EventManagerTest.Create_InvalidEvent_ThrowsAndDoesNotNotify                        tst_event_manager.cpp 44
  PASS   EventManagerTest.Update_StartTimeChanges_ReindexesChronologicalOrderAndNotifies    tst_event_manager.cpp 60
  PASS   EventManagerTest.SearchByTitle_IsCaseInsensitive_AndEmptyNeedleReturnsAll          tst_event_manager.cpp 97
         Test execution took 0 ms total.                                                    tst_event_manager.cpp 97
PASS   Running tests for "F:/git/oop-assignment-2-bryan-jang-daniel-yuan/code/build/Desktop_Qt_6_10_1_MinGW_64_bit-Debug".   10 ms
  PASS   1/7 Test #1: ValidateTest.NonEmpty_TrimsWhitespace ..................................... Passed   0.01 sec
         Test execution took 0.01 sec.
  PASS   2/7 Test #2: ValidateTest.Chronological_AllowsNoEndAndEndAfterStart ............... Passed   0.01 sec
         Test execution took 0.01 sec.
  PASS   3/7 Test #3: ValidateTest.Event_FailsWhenEndBeforeStart_AndWhenTitleEmpty ........ Passed   0.01 sec
         Test execution took 0.01 sec.
  PASS   4/7 Test #4: EventManagerTest.Create_ValidEvent_AssignsIdIfMissingAndNotifiesListeners ..... Passed   0.01 sec
         Test execution took 0.01 sec.
  PASS   5/7 Test #5: EventManagerTest.Create_InvalidEvent_ThrowsAndDoesNotNotify ........ Passed   0.01 sec
         Test execution took 0.01 sec.
  PASS   6/7 Test #6: EventManagerTest.Update_StartTimeChanges_ReindexesChronologicalOrderAndNotifies ... Passed   0.01 sec
         Test execution took 0.01 sec.
  PASS   7/7 Test #7: EventManagerTest.SearchByTitle_IsCaseInsensitive_AndEmptyNeedleReturnsAll ..... Passed   0.01 sec
         Test execution took 0.01 sec.
  INFO   100% tests passed, 0 tests failed out of 7
  INFO   Total Test time (real) =   0.08 sec
```

All unit tests passed successfully.

- **14 tests passed**
- **0 tests failed**
- Total runtime was under 100 milliseconds

This confirms that the core functionality behaves as expected and that no regressions were introduced.

7. Edge Case Handling

Several edge cases were tested to make sure the application is robust:

- Titles with only spaces are rejected
- Events without an end time are allowed
- Events with an end time before the start time are rejected
- Invalid events do not trigger listener notifications
- Searching with an empty string returns all events

Handling these cases helps prevent incorrect data from entering the system and improves overall reliability.

8. Summary

Overall, unit testing with GoogleTest and Google Mock helped verify both correctness and behavior of the core logic. By testing validation rules, event management, and observer notifications, the application is more reliable and easier to maintain.

Reflection

Challenges Faced

One of the main challenges was coordinating communication between the user interface and the underlying event data while keeping the system modular. Managing time representations was also non-trivial, as the application needed to convert between Qt's **QDateTime** types and **std::chrono** time points without introducing errors. Additionally, handling optional event fields (such as end time, location, and notes) required careful validation to avoid inconsistent or incomplete event states.

Another challenge was correctly mapping user interface selections (such as a clicked table row) back to the corresponding event in the data model, which required storing and retrieving event identifiers in the UI layer.

Trade-offs Made

A key trade-off was choosing Qt's **QStandardItemModel** instead of a fully custom model. While a custom model could offer finer control and potentially better performance for very large datasets, the standard model significantly reduced implementation complexity and development time.
Similarly, using modal dialogs (**EventDialog**) simplified input validation and user flow but limited advanced interactions such as inline editing or multi-event editing.

The decision to display event notes in a popup rather than a dedicated panel was another trade-off, prioritizing simplicity and clarity over a richer but more complex interface.

Lessons Learned

This project reinforced the importance of separating concerns between the UI, application logic, and data management. Applying design patterns such as Observer and Model–View early helped prevent tightly coupled code and made later feature additions (such as automatic UI refresh and note display on event click) much easier to implement.

The project also highlighted the value of leveraging framework-provided patterns and tools instead of reinventing solutions. Using Qt's model/view architecture and signal-slot mechanism resulted in cleaner, more maintainable code and reduced the likelihood of subtle UI synchronization bugs.

Overall, the experience demonstrated that thoughtful architectural decisions early in development can greatly simplify future changes and improve the reliability of the system.

Video Demo

https://youtube.com/playlist?list=PL-cZhLrfWjNLmI-d139UeBgsUapmr_SE6&si=z6U3J4t1wbXfeEv5